COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	02/10/2026	**DATE SUBMITTED:**	02/10/2026
NAME:	Guarin, Jan MIkaela C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

A1

```
A.1 Create a Graph
Vertices: A, B, C, D, E, F
```

```
// Show the vertices of the graph
void showVertices() {
    cout << "Vertices: ";

    for (char v = 'A'; v <= 'F'; v++) {
        cout << v;

        if (v != 'F') {
            cout << ", ";
        }
    } <- #23-29 for (char v = 'A'; v <= 'F

    cout << endl;
} <- #20-32 void showVertices()
```

A2

```
A.2 Adjacency Matrix

Adjacency Matrix

  A B C D E F
A 0 2 4 0 0 0
B 0 0 0 0 3 0
C 0 0 0 2 0 0
D 0 4 0 0 0 0
E 0 0 0 3 0 0
F 0 0 0 0 0 0
```

```
// Create the adjacency matrix
void createMatrix(Edge edges[], int totalEdges, int matrix[][SIZE]) {

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            matrix[i][j] = 0;
        }
    } <- #38-42 for (int i = 0; i < SIZE; i++)

    for (int i = 0; i < totalEdges; i++) {
        matrix[edges[i].from][edges[i].to] = edges[i].weight;
    }
} <- #36-47 void createMatrix(Edge edges[], int totalEdges, int matrix
```

```
/ Display the adjacency matrix
oid showMatrix(int matrix[][SIZE]) {

   cout << "\nAdjacency Matrix\n\n";

   cout << "  ";

   for (char v = 'A'; v <= 'F'; v++) {
       cout << v << " ";
   }

   cout << endl;

   for (int i = 0; i < SIZE; i++) {

       cout << char('A' + i) << " ";

       for (int j = 0; j < SIZE; j++) {
           cout << matrix[i][j] << " ";
       }
```

A3

```
Adjacency List
(Start Vertex, End Vertex, Weight)
(A, B, 2)
(A, C, 4)
(B, E, 3)
(C, D, 2)
(D, B, 4)
(E, D, 3)
```

```
// Add an item to the adjacency list
void addNode(ListNode*& head, char vertex, int weight)

    ListNode* newItem = new ListNode;

    newItem->vertex = vertex;
    newItem->weight = weight;
    newItem->next = NULL;

    if (head == NULL) {
        head = newItem;
    }
    else {
        ListNode* current = head;

        while (current->next != NULL) {
            current = current->next;
        }

        current->next = newItem;
    } <- #88-96 else
```

```
// Display the adjacency list
void showList(ListNode* graph[]) {

    cout << "\nAdjacency List\n";
    cout << "(Start Vertex, End Vertex, Weight)" << endl;

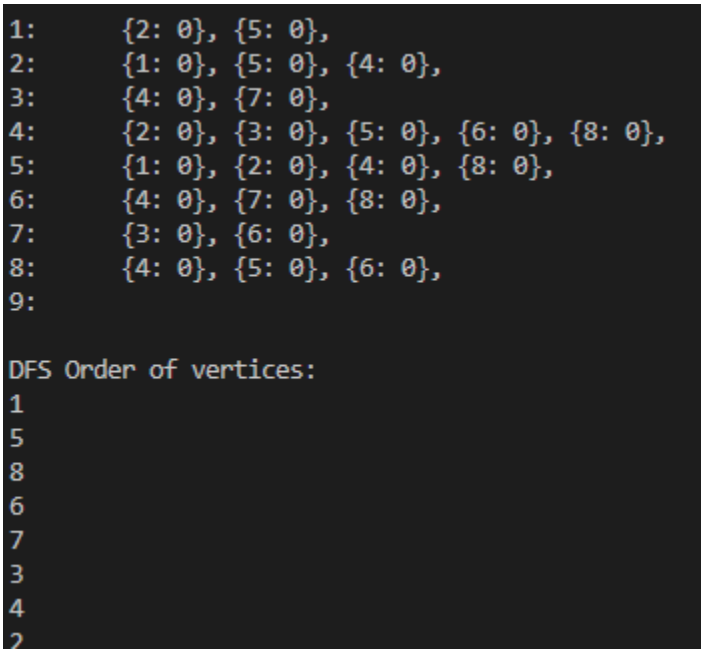
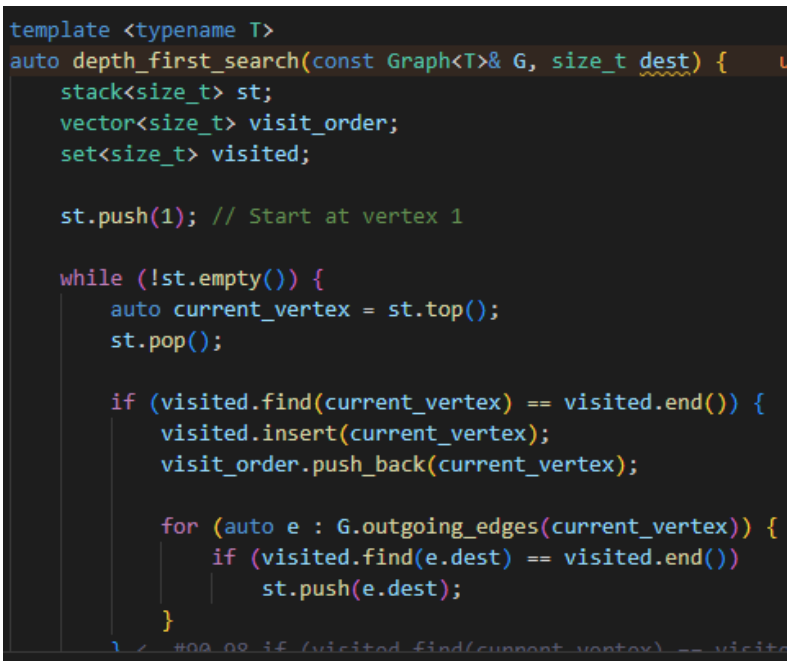
    for (int i = 0; i < SIZE; i++) {

        ListNode* current = graph[i];

        while (current != NULL) {

            cout << "("
                 << char('A' + i) << ", "
                 << current->vertex << ", "
                 << current->weight << ")"
                 << endl;

            current = current->next;
        } <- #110-119 while (current != NULL)
    } <- #106-120 for (int i = 0; i < SIZE; i++)
```

ILO B1	 

ILO B2

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
9:

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

```cpp
template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest) {
    queue<size_t> q;
    vector<size_t> visit_order;
    set<size_t> visited;

    q.push(1); // Start at vertex 1

    while (!q.empty()) {
        auto current_vertex = q.front();
        q.pop();

        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

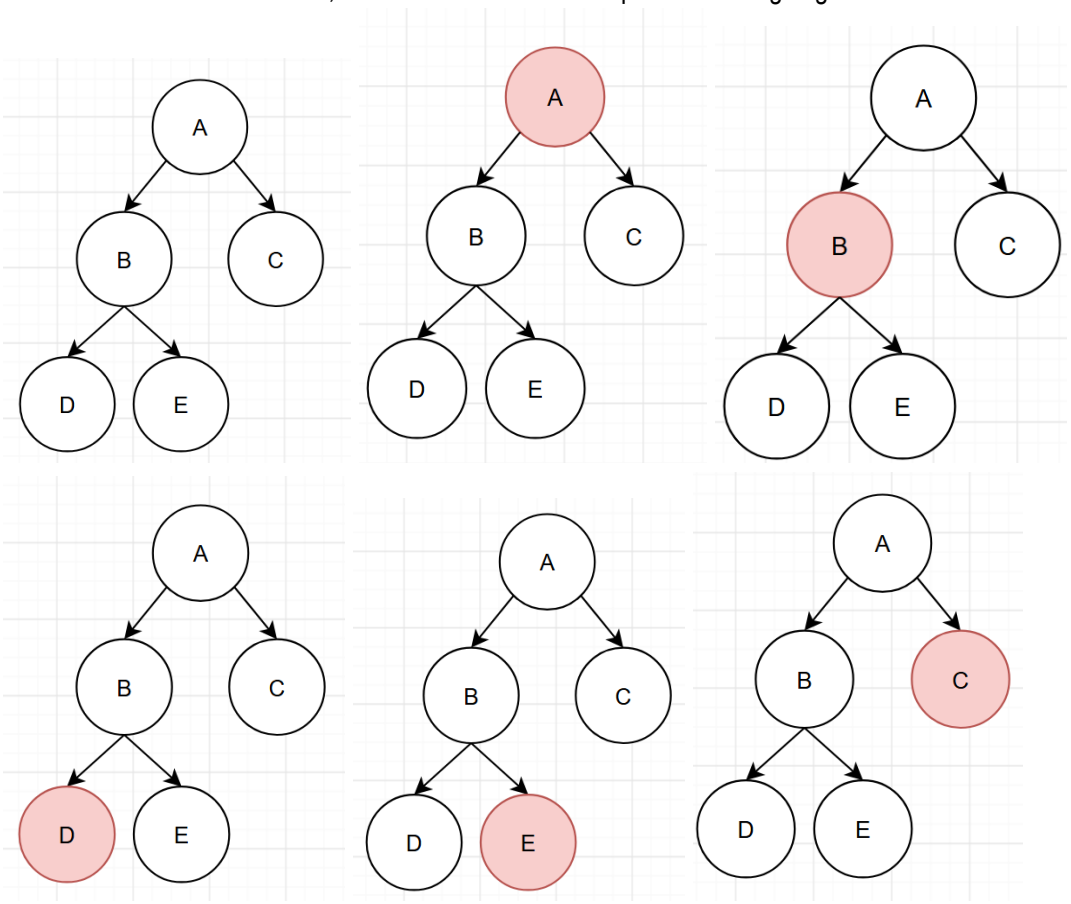
            for (auto e : G.outgoing_edges(current_vertex)) {
                if (visited.find(e.dest) == visited.end())
                    q.push(e.dest);
            }
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
    - **Depth First Search (DFS)** would be the most helpful algorithm because **it explores one vertex and continues to the deepest possible vertex before going back**. When it reaches a point where there are no more unvisited vertices, it backtracks and explores another path.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
    - The equivalent of DFS in tree traversal is **Preorder Traversal**. In both methods, the current node is visited first, then its child nodes are explored before going back to other branches.



The order of visiting is: **A → B → D → E → C**

PSEUDOCODE:

```
DFS(vertex):
   mark vertex as visited
   for each neighbor:
      if not visited:
         DFS(neighbor)
```

TREE PREORDER:

```
Preorder(node):
   visit node
   Preorder(left)
   Preorder(right)
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
    - The data structure used to implement Breadth First Search (BFS) is a queue. A queue follows FIFO (First In, First Out), so the vertex that enters the queue first is also the first one to be

processed.

4. How many times can a node be visited in the BFS?
    - A node in BFS can normally be visited only once when a visited list or set is used. Once a node is marked as visited, the algorithm will not process it again, which helps avoid repeated visits and infinite loops in graphs with cycles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.